CAP599:

Next Gen Project - I

Class Assessment - II Report

On

Workforce Skill Gap Analyzer

Submitted by

(Anubhav Singh, Manish Kumar, Abhishek Kumar, Karan Bisht)

(12501540, 12516115, 12527809, 1250146).

IN

(-------Add Area of research/ project category here-----)

Under the Supervision of

Mrs. Yamini Bhardwaj, UID-30979

Assistant Professor, SCA

Department of Computer Applications

LOVELY PROFESSIONAL UNIVERSITY

PUNJAB

(April, 2026)

1. Introduction

The Workforce Skill Gap Analyzer is a responsive, frontend-only web application designed to bridge the growing gap between employee competencies and the evolving demands of modern job roles. Built entirely with HTML5, CSS3, and Vanilla JavaScript, the system operates as a Single Page Application (SPA) without requiring any server-side infrastructure, making it highly accessible and easy to deploy.

Organizations today face significant challenges in identifying where their workforce lacks the skills needed to fulfill specific roles. Traditional methods of skill assessment are often manual, time-consuming, and subjective. This project automates the process of skill gap analysis by allowing administrators and users to input employee skill sets, define job role requirements, and instantly compute gap metrics with actionable recommendations.

The application leverages browser-based Local Storage for persistent data management, the Canvas API for visual chart rendering, and third-party libraries such as pdf.js and mammoth.js for automated skill extraction from uploaded resumes. The system also features role-based access control, light/dark mode theming, and CSV report export — making it a comprehensive tool for HR analysts, project managers, and educational institutions alike.

Key highlights of the project:

- No backend or server required — runs entirely in the browser
- Role-based login system for Admin and Normal Users
- Automated resume parsing for skill extraction (PDF and DOCX)
- Visual gap analysis with progress bars and pie charts
- CSV export capability for reporting and HR auditing
- Fully responsive design with dark mode support

2. Team Information

The project was developed by a team of three members; each assigned a distinct role to ensure efficient division of labor and comprehensive coverage of all system components.

Member	Role	Responsibilities
Anubhav Singh	Frontend Developer, Report writing	UI design, CSS theming, responsive layout, dark/light mode, developer slider section, footer
Manish Kumar	Backend / Logic Developer	Gap analysis algorithm, role-based authentication, resume parsing (pdf.js, mammoth.js), Canvas API charts
Abhishek Kumar	Database / Storage Designer	Local Storage schema design, CRUD operations, CSV export logic, data persistence and state management
Karan Bisht	Database / Storage Designer	Local Storage schema design, CRUD operations, CSV export logic, data persistence and state management

Each team member collaborated on system design, testing, and documentation phases to ensure a cohesive final product.

3. Problem Statement

In today's rapidly evolving technology landscape, organizations and educational institutions face a critical challenge: employees and students often possess skill sets that do not align with the requirements of their current or target job roles. This misalignment — commonly referred to as a "skill gap" — leads to reduced productivity, increased training costs, missed project deadlines, and poor talent utilization.

The core problems that this project aims to solve are:

1. Lack of Visibility: Managers and HR teams have no clear, data-driven view of which employees are under-skilled for their assigned roles, and by how much.
2. Manual Assessment Overhead: Existing skill tracking is often done via spreadsheets or paper-based forms, which are inefficient, error-prone, and difficult to update.
3. No Actionable Feedback: Even when skill gaps are identified, employees are rarely provided with targeted course recommendations or learning paths to close those gaps.
4. Resume Data Underutilization: Employee resumes contain rich skill information, but this data is rarely extracted and structured for comparative analysis.
5. Access Control Gap: In organizational settings, different stakeholders (admins vs. regular users) need different levels of access, which simple tools typically do not support.

The Workforce Skill Gap Analyzer addresses all of the above by providing a structured, automated, and visually intuitive platform that empowers organizations to make data-driven decisions about workforce development.

4. Project Description

The Workforce Skill Gap Analyzer is a Single Page Application (SPA) built using pure web technologies. It provides a complete workflow from employee skill registration to job role comparison, gap computation, and course recommendations — all within a single browser window without any page reloads.

4.1 Objectives

- Provide a structured platform to register and manage employee skill sets
- Allow definition of job role requirements with skill names and proficiency levels
- Automate the computation of skill gaps using a weighted scoring algorithm
- Visualize gaps through progress bars and pie charts for intuitive understanding
- Suggest relevant online courses for each missing or under-developed skill
- Generate exportable reports in CSV format for HR and management use
- Enforce role-based access control to separate admin and user capabilities

4.2 Key Features

Role-Based Authentication

The system supports two user roles — Admin and Normal User — each with distinct credentials and access privileges. Login validation enforces email format, password strength, age verification (18+), and role-specific credential matching.

Employee Skills Module

Users can manually input employee skill names and proficiency levels (on a numeric scale). Alternatively, they can upload a PDF or DOCX resume, which is parsed using pdf.js and mammoth.js to automatically extract and populate skill fields based on a common skill dictionary.

Job Role Module

Admins can define job roles and their required skills with expected proficiency levels. Sample job profiles (e.g., Full Stack Developer, Data Analyst, UX Designer) can be loaded with one click to speed up configuration.

Gap Analysis Module

The gap engine compares each required skill against the employee's profile. It computes a gap score, gap percentage, and identifies missing skills. Results are displayed visually via a progress bar and a Canvas-rendered pie chart. After analysis, tailored course suggestions with clickable links are shown for each gap area.

Reports Module

A tabular report view lists all analyzed employee-role combinations with gap percentage and date. Multi-color role coverage bars provide a visual summary. Reports can be exported to CSV for external use.

UI/UX Features

The interface supports light and dark mode toggling, hover interactions, a swipeable developer profile slider, and a responsive footer with quick links, a newsletter subscription form, and a feedback form.

4.3 Working of the System

6. User logs in with role-specific credentials (Admin or Normal User)
7. Admin configures job roles and required skill levels
8. Employee skills are added manually or extracted from an uploaded resume
9. The gap analysis engine processes the data and computes metrics
10. Results are displayed with visualizations and course recommendations
11. Reports are saved to LocalStorage and can be exported as CSV

5. System Design

5.1 Data Flow Diagram (DFD)

Figure 1: DFD Level 0 — Context Diagram

The Level 0 DFD (Context Diagram) provides a high-level overview of the system. It shows the Workforce Skill Gap Analyzer as a single process that interacts with two external entities: the Admin and the Normal User. The system receives skill and role data as input and produces gap analysis reports and course recommendations as output.

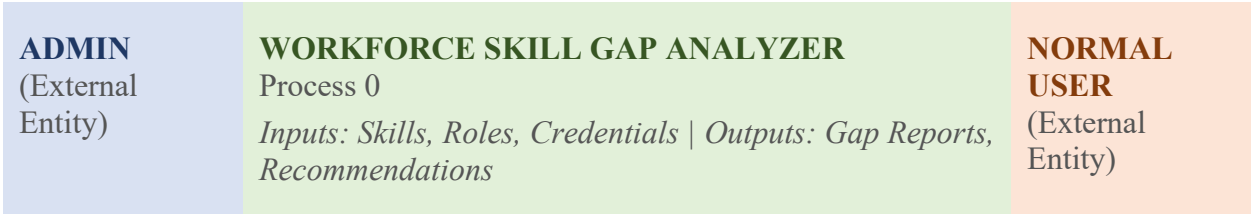

Figure 1: DFD Level 0 — Context Diagram (Workforce Skill Gap Analyzer)

Figure 2: DFD Level 1 — Detailed Process Diagram

The Level 1 DFD expands the single process from Level 0 into its major sub-processes. It shows how data flows between the authentication module, skill management, job role configuration, gap analysis engine, and the reporting system.

Process No.	Process Name	Input	Output	Data Store
P1	Authentication	Email, Password, Role	Session Token / Error	DS1: User Credentials
P2	Employee Skill Management	Skill Name, Level, Resume File	Employee Skill Profile	DS2: Employee Skills

Process No.	Process Name	Input	Output	Data Store
P3	Job Role Management	Role Name, Required Skills/Levels	Job Role Profile	DS3: Job Roles
P4	Gap Analysis Engine	Employee Profile + Job Role Profile	Gap Score, Gap %, Missing Skills	DS2 + DS3
P5	Recommendation Engine	Missing Skills List	Course Links	Embedded Dictionary
P6	Reports Module	Gap Analysis Results	Report Table, CSV Export	DS4: Reports

Figure 2: DFD Level 1 — Sub-Process Data Flow Diagram

5.2 ER Diagram

The Entity-Relationship (ER) Diagram models the logical data structure of the application. Since the system uses browser Local Storage (key-value storage), the ER diagram represents the logical entities and their relationships as they would appear in a relational database model.

Entity	Attributes	Primary Key	Relationships
User	userId, name, email, password, role, age	userId	1 User → Many Reports
Employee	employeeId, name, email, department, skills[]	employeeId	1 Employee → Many Reports
Skill	skillId, skillName, level (1-10), source (manual/resume)	skillId	Many Skills → 1 Employee
JobRole	roleId, roleName, requiredSkills[]	roleId	1 JobRole → Many Reports
RequiredSkill	reqSkillId, skillName, requiredLevel	reqSkillId	Many RequiredSkills → 1 JobRole
GapReport	reportId, employeeId, roleId, gapScore, gapPercent, date	reportId	Joins Employee + JobRole
Course	courseId, skillName, courseTitle, courseURL	courseId	Many Courses → 1 Skill (gap)

Figure 3: ER Diagram — Logical Entity-Relationship Model

Key Relationships:

- A User (Admin) can manage many Employees and JobRoles.
- Each Employee has many Skills, recorded either manually or via resume parsing.
- Each JobRole has many RequiredSkills with defined proficiency levels.
- A GapReport links one Employee to one JobRole and records gap metrics.
- Courses are mapped to individual skill gaps identified in the analysis.

6. Database Design

The Workforce Skill Gap Analyzer uses browser LocalStorage as its persistence layer. All data is stored as serialized JSON strings under structured key names. Below are the schema designs for each logical data store.

6.1 User Credentials Schema (DS1)

Field	Type	Description
role	String	"admin" or "user" — determines access level
email	String	Login email address (validated format)
password	String	Password (validated against role-specific credentials)
age	Number	User age — must be 18 or above to access the system

6.2 Employee Skills Schema (DS2)

Field	Type	Description
employeeId	String	Unique identifier for the employee
name	String	Full name of the employee
email	String	Email address of the employee
department	String	Department or team the employee belongs to
skills	Array<Object>	List of skill objects: { skillName: String, level: Number (1-10) }
source	String	"manual" if entered by user, "resume" if extracted via parsing

6.3 Job Role Schema (DS3)

Field	Type	Description
roleId	String	Unique identifier for the job role
roleName	String	Title of the job role (e.g., Full Stack Developer)
requiredSkills	Array<Object>	List of required skills: {skillName: String, requiredLevel: Number}

6.4 Gap Report Schema (DS4)

Field	Type	Description
reportId	String	Unique identifier for the report entry
employeeName	String	Name of the analyzed employee
roleName	String	Job role analyzed against
gapScore	Number	Total numeric gap score (sum of all individual gaps)
gapPercentage	Number	Gap expressed as percentage of maximum possible score
missingSkills	Array<String>	List of skills employee does not possess at all
date	String	ISO date string of when the analysis was performed

6.5 Local Storage Key Naming Convention

LocalStorage Key	Stores	Format
wsa_employees	All employee records	JSON Array of Employee objects
wsa_jobroles	All job role definitions	JSON Array of JobRole objects
wsa_reports	All generated gap reports	JSON Array of GapReport objects
wsa_theme	User's UI theme preference	String: "light" or "dark"
wsa_session	Current logged-in user role	String: "admin" or "user"

7. Screenshots

The following section describes the major screens of the Workforce Skill Gap Analyzer. Actual UI screenshots should be inserted in place of the placeholder descriptions below when preparing the final submitted report.

Figure	Screen / Module	Description
Figure 4	Login Page	Role selector dropdown (Admin/User), email field, password field, age field. Validation errors displayed inline. Submit triggers credential check.
Figure 5	Admin Dashboard	Navigation tabs visible to admin: Employee Skills, Job Roles, Gap Analysis, Reports. Dark mode toggle in header.
Figure 6	Employee Skills Module	Form to add employee name, department, skills (name + level). Resume upload button triggers pdf.js/mammoth.js extraction. Auto-fill of detected skills shown.
Figure 7	Job Role Module	Interface to define role name and required skills/levels. 'Load Sample Profile' button to pre-populate with example roles like Data Analyst or UX Designer.
Figure 8	Gap Analysis Results	Analysis output showing gap score, gap percentage, progress bar, and Canvas pie chart. Missing skills listed below with color coding.
Figure 9	Course Recommendations	Auto-generated list of recommended online courses for each missing skill. Clickable links open course pages in a new tab.
Figure 10	Reports Module	Tabular view of all historical analyses with employee name, role, gap %, and date. Multi-color role coverage bars. CSV Export button at top.
Figure 11	Dark Mode UI	Full application shown in dark theme. All components — forms, tables, charts — adapt to the dark color palette automatically.

Table of Screenshots — Insert corresponding UI captures when submitting

Fig 4. Login Page

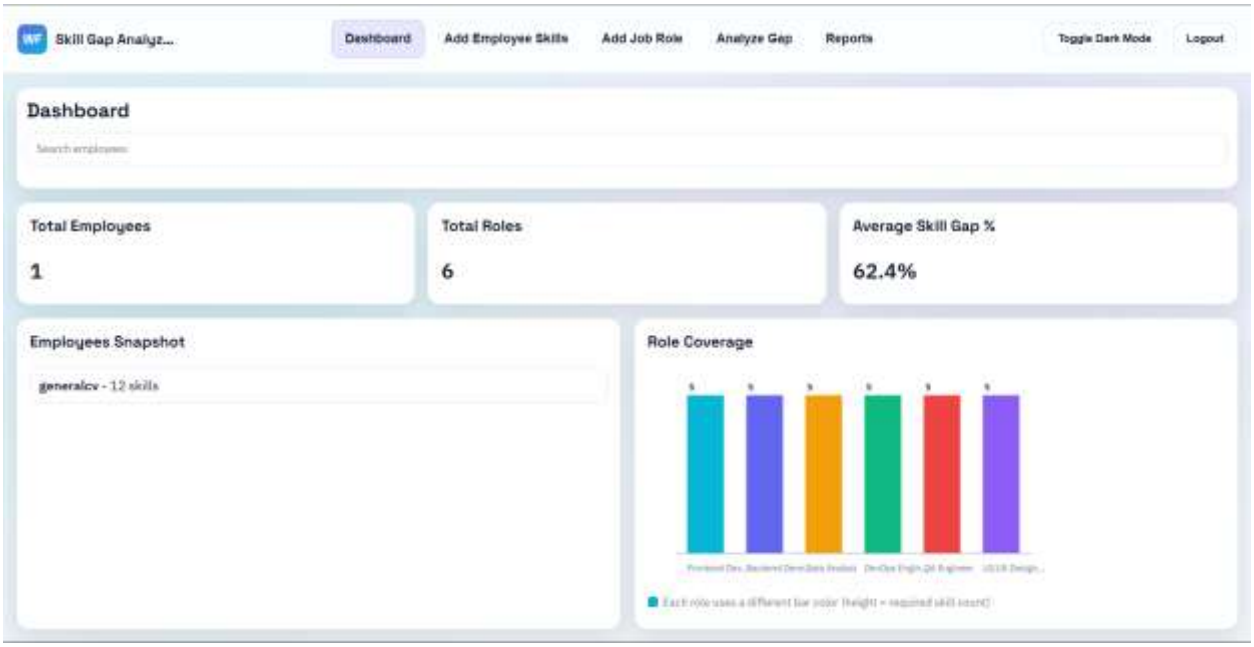

Fig 5. Admin Dashboard

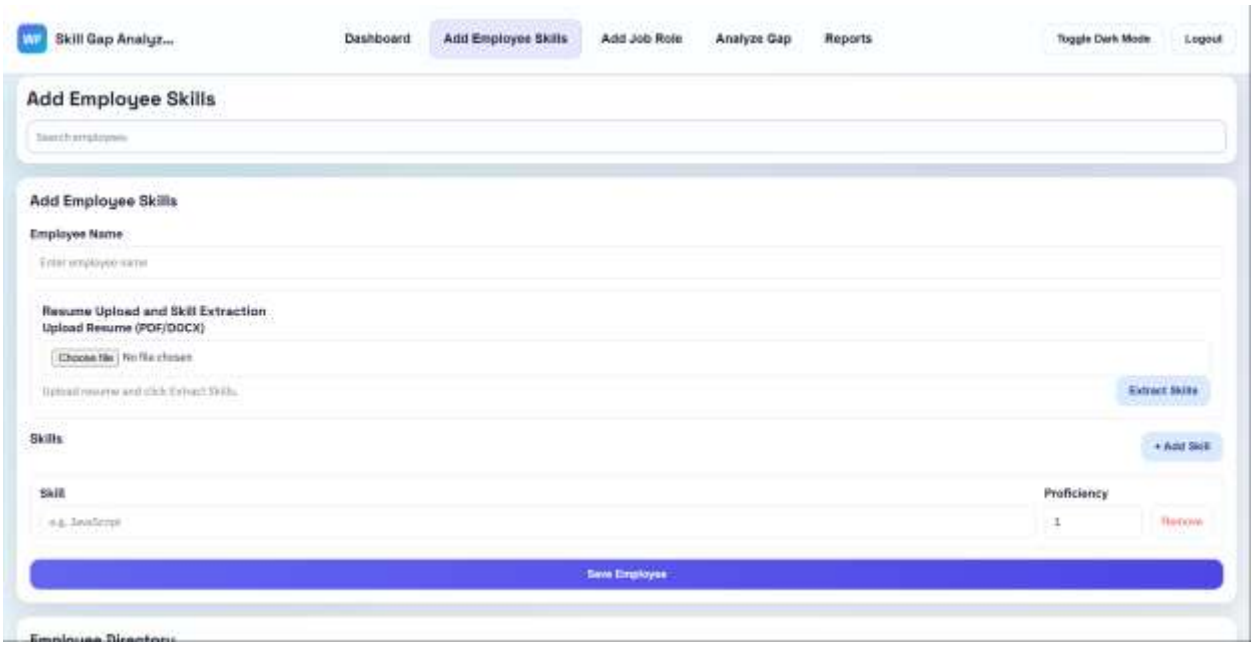

Fig 6. Employee Skills Module

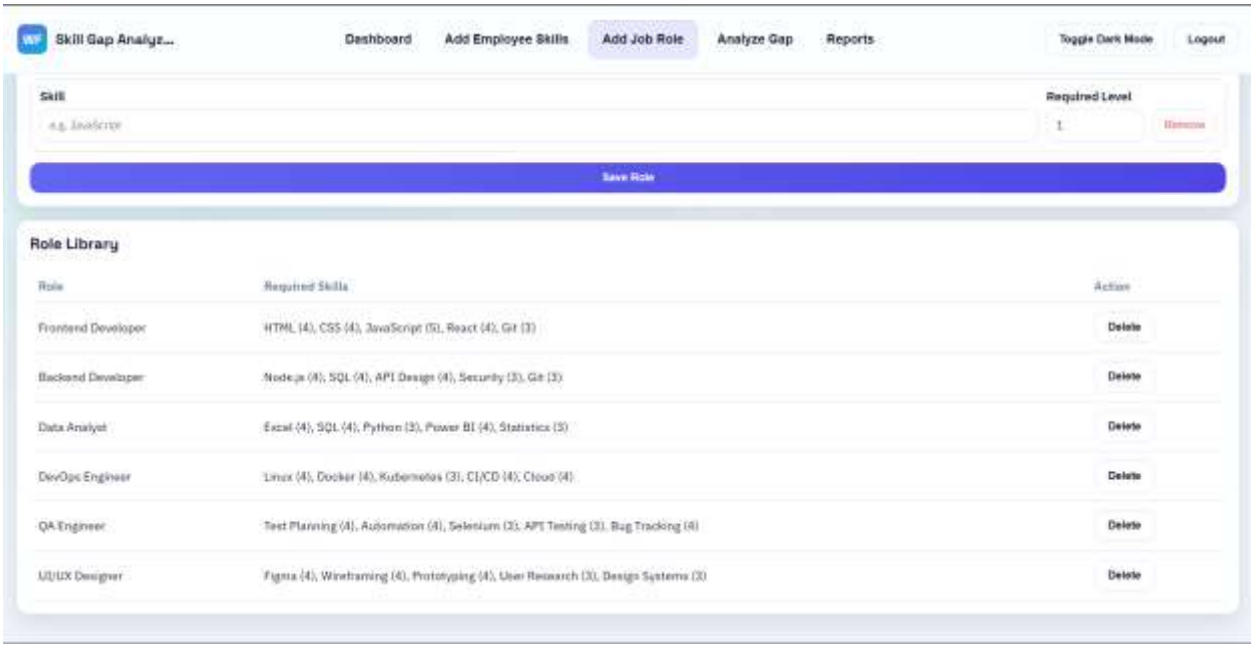

Fig 7. Job Role Module

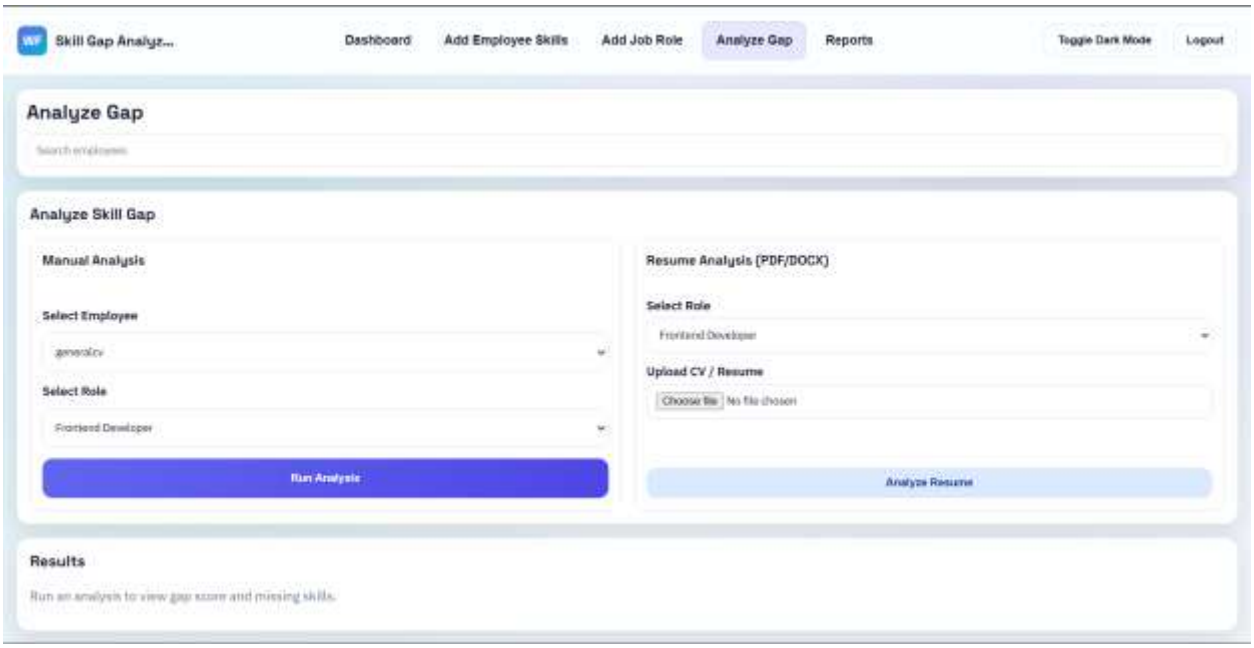

Fig 8. Gap Analysis Results

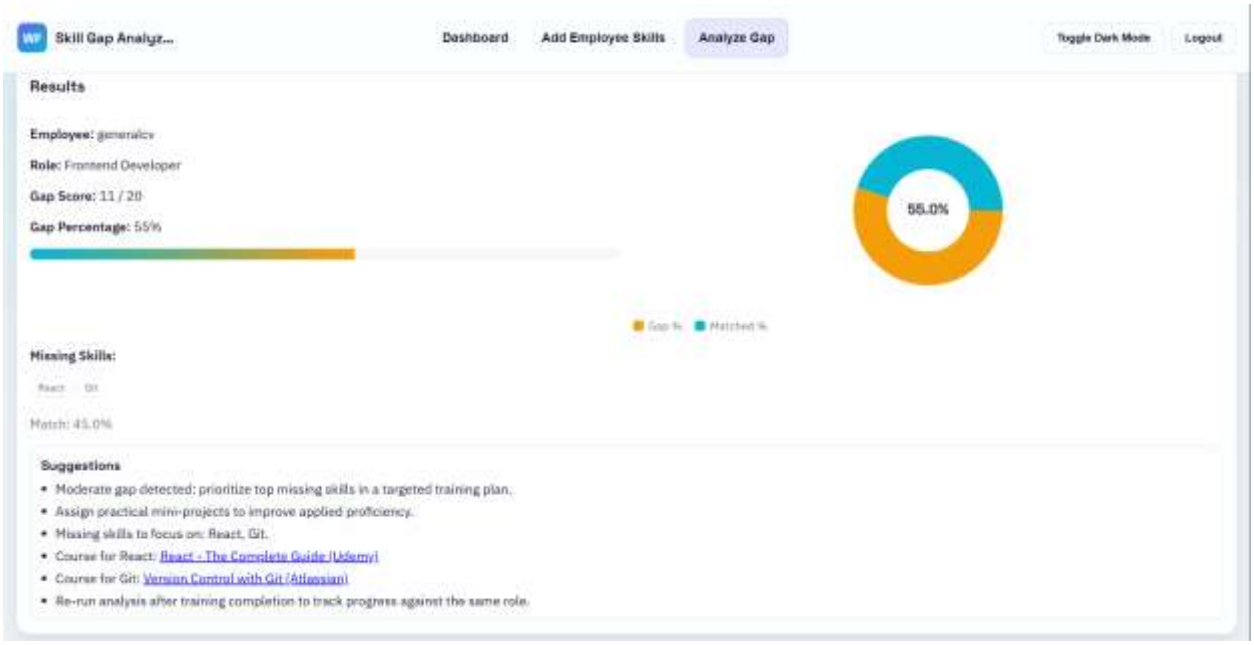

Fig 9. Course Recommendations

Fig. 10 Reports Module

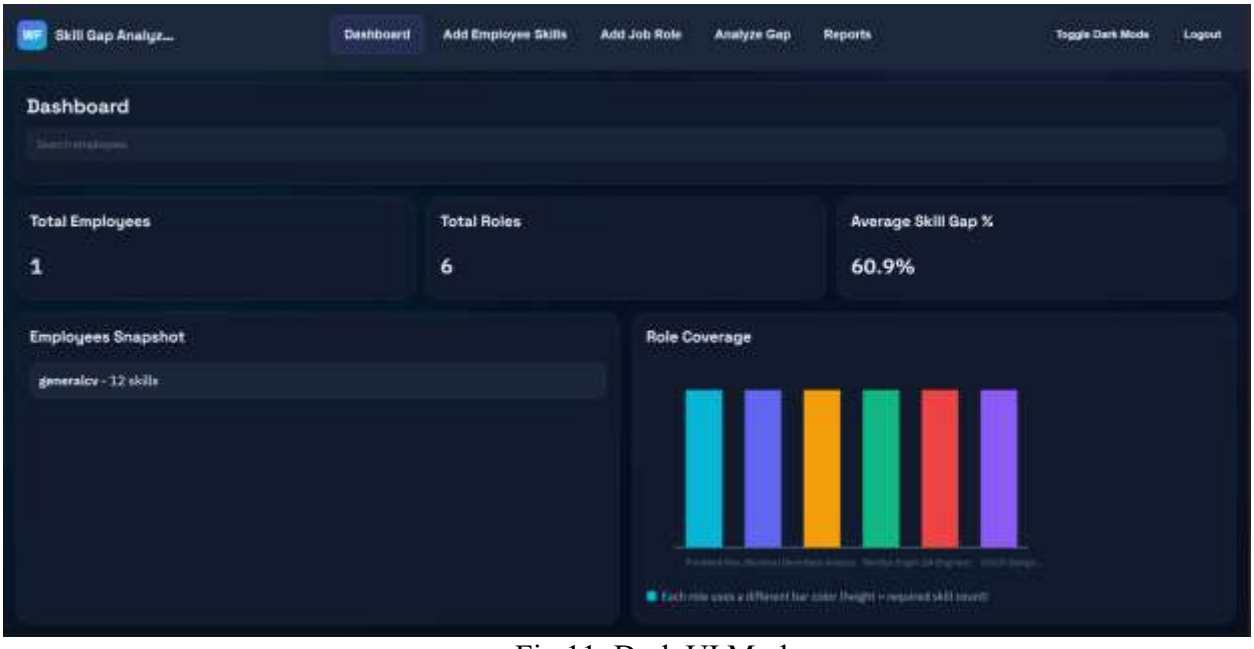

Fig 11. Dark UI Mode

8. Software Requirement Specification (SRS)

8.1 Functional Requirements

FR-1: User Authentication

- The system shall provide a login form with role selection (Admin / Normal User).
- Email must match a valid format (e.g., user@gmail.com).
- Age input must be validated to reject users below 18.
- Credentials must be verified against role-specific hardcoded values.
- Admin credentials: admin@gmail.com / Admin@123.
- User credentials: user@gmail.com / User@123.

FR-2: Role-Based Access Control

- Admin shall have access to all modules: Employee Skills, Job Roles, Gap Analysis, Reports.
- Normal Users shall have restricted access based on system configuration.
- Session state shall persist in Local Storage until logout.

FR-3: Employee Skills Management

- Users shall be able to manually add employee records with name, email, department, and skills.
- Each skill entry shall have a name and a numeric proficiency level (1–10).
- Users shall be able to edit existing employee skill records.
- Users shall be able to upload PDF or DOCX resumes for skill extraction.
- The system shall extract skills from resumes using pdf.js (PDF) and mammoth.js (DOCX).
- Extracted skills shall be auto-filled into the employee skill entry form.

FR-4: Job Role Management

- Admin shall be able to create job role profiles with a role name and required skills.
- Each required skill entry shall include the skill name and a required proficiency level.
- Admin shall be able to load pre-built sample job profiles.

FR-5: Gap Analysis

- The system shall compute a gap score for each required skill using the defined formula.
- If the employee has the skill: gap = max (0, requiredLevel - employeeLevel).
- If the employee lacks the skill: gap = requiredLevel.
- The system shall compute totalGapScore, maxScore, and gapPercentage.
- Results shall be displayed with a progress bar and a Canvas API pie chart.
- Missing skills shall be listed with course recommendations and clickable links.

FR-6: Reports

- The system shall generate a tabular report of all analyses with employee, role, gap %, and date.
- Reports shall display multi-color role coverage bars for visual comparison.

- Users shall be able to export all report data as a CSV file.

FR-7: UI/UX

- The system shall support light and dark mode toggling.
- All navigation shall function as a Single Page Application without full page reloads.
- A developer profile slider with swipe and button navigation shall be present.
- The footer shall contain quick links, a newsletter subscription form, and a feedback form.

8.2 Non-Functional Requirements

Performance

- The application shall load fully within 3 seconds on a standard broadband connection.
- Gap analysis computation shall complete within 1 second for up to 50 skills.
- Resume parsing for standard PDF/DOCX files shall complete within 5 seconds.
- The Canvas API chart shall render within 500ms of analysis completion.

Security

- Session data shall be stored in LocalStorage and cleared upon logout.
- Input fields shall validate all data before processing to prevent XSS injection through form values.
- No sensitive data shall be transmitted to any external server.
- Role-based access shall prevent normal users from accessing admin-only sections.

Scalability

- The system shall support up to 100 employee records in LocalStorage without performance degradation.
- The gap analysis algorithm is O(n) in the number of skills, ensuring linear scalability.
- The CSV export function shall handle up to 500 report entries efficiently.

Usability

- The interface shall be fully responsive and functional on mobile, tablet, and desktop devices.
- Dark mode shall be available across all sections for reduced eye strain.
- All error messages shall be descriptive and displayed inline near the relevant input field.
- Course recommendation links shall open in a new browser tab for uninterrupted workflow.
- The developer slider shall support both touch/swipe (mobile) and button navigation (desktop).

Reliability

- All data shall persist across browser sessions using LocalStorage.

- Clearing browser storage is the only way to reset application data — a warning shall be visible.
- The system shall gracefully handle scanned/image-only PDFs by displaying an extraction failure notice.

Compatibility

- The application shall be compatible with Chrome, Firefox, Edge, and Safari (latest versions).
- The system shall function without any backend server — a static file server (e.g., Five Server) or direct file opening is sufficient.

9. Testing

The application was tested using manual black-box testing methodology. Test cases were designed to verify each functional requirement and edge case.

9.1 Test Cases — Authentication Module

TC#	Test Description	Input	Expected Output	Status
TC-01	Valid Admin Login	admin@gmail.com Admin@123 / Role: Admin	Login successful, admin dashboard shown	Pass
TC-02	Valid User Login	user@gmail.com User@123 / Role: User	Login successful, user dashboard shown	Pass
TC-03	Invalid Email Format	Not an email / Admin@123	Inline error: Invalid email format	Pass
TC-04	Wrong Password	admin@gmail.com wrong pass	Error: Invalid credentials	Pass
TC-05	Age Below 18	admin@gmail.com/ Admin@123 / Age: 16	Error: Must be 18 or older	Pass
TC-06	Role Mismatch	admin@gmail.com// Admin@123 / Role: User	Error: Credentials do not match selected role	Pass

9.2 Test Cases — Employee Skills Module

TC#	Test Description	Input	Expected Output	Status
TC-07	Add Employee Manually	Name, email, 3 skills with levels	Employee saved to LocalStorage, appears in list	Pass
TC-08	Edit Existing Employee	Modify skill level for existing employee	Updated record saved correctly	Pass
TC-09	Upload PDF Resume	Standard text-based PDF with skills	Skills extracted and auto-filled into form	Pass
TC-10	Upload DOCX Resume	DOCX file with skill keywords	Skills extracted using mammoth.js, form auto-filled	Pass

TC#	Test Description	Input	Expected Output	Status
TC-11	Upload Scanned PDF	Image-only PDF (no text layer)	Notice: Unable to extract text from scanned PDF	Pass
TC-12	Unsupported File Type	.txt file upload attempt	Error: Only PDF and DOCX supported	Pass

9.3 Test Cases — Gap Analysis Module

TC#	Test Description	Input	Expected Output	Status
TC-13	Full Skill Match	Employee has all required skills at or above required levels	Gap Score: 0, Gap %: 0%, No missing skills	Pass
TC-14	Partial Skill Match	Employee has some skills below required levels	Correct partial gap score and percentage calculated	Pass
TC-15	Complete Skill Mismatch	Employee has none of the required skills	Gap % = 100%, all skills listed as missing	Pass
TC-16	Chart Rendering	Any valid gap analysis result	Progress bar and Canvas pie chart render correctly	Pass
TC-17	Course Recommendations	Analysis with missing skills	Relevant courses with clickable links displayed	Pass

9.4 Test Cases — Reports Module

TC#	Test Description	Input	Expected Output	Status
TC-18	Report Generation	Complete gap analysis performed	New row added to report table with correct data	Pass
TC-19	CSV Export	Click Export CSV with 3 existing reports	CSV file downloaded with correct headers and data	Pass
TC-20	Report Persistence	Close and reopen browser	All reports still present in LocalStorage	Pass

9.5 Test Cases — UI/UX

TC#	Test Description	Input / Action	Expected Output	Status
TC-21	Dark Mode Toggle	Click dark mode switch	All UI components switch to dark theme	Pass
TC-22	Responsive Design	Resize to mobile viewport (375px)	Layout adapts, no horizontal overflow, forms usable	Pass
TC-23	Developer Slider	Click left/right buttons or swipe	Slider transitions to correct developer card	Pass
TC-24	Newsletter Form	Enter email and submit	Subscription acknowledgment shown	Pass
TC-25	Feedback Form	Enter feedback text and submit	Feedback confirmation message displayed	Pass

10. References

Books

1. Flanagan, D. (2020). JavaScript: The Definitive Guide (7th ed.). O'Reilly Media.
2. Duckett, J. (2011). HTML and CSS: Design and Build Websites. John Wiley & Sons.
3. Haverbeke, M. (2018). Eloquent JavaScript: A Modern Introduction to Programming (3rd ed.). No Starch Press.

Online Documentation

4. Mozilla Developer Network (MDN). (2024). Web APIs — LocalStorage. https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
5. Mozilla Developer Network (MDN). (2024). Canvas API — CanvasRenderingContext2D. https://developer.mozilla.org/en-US/docs/Web/API/CanvasRenderingContext2D
6. pdf.js — Mozilla. (2024). PDF.js: A General-Purpose, Web Standards-Based Platform for Parsing and Rendering PDFs. https://mozilla.github.io/pdf.js/
7. mammoth.js. (2024). Convert Word documents (.docx files) to HTML. https://github.com/mwilliamson/mammoth.js

Research Papers

8. Acemoglu, D., & Restrepo, P. (2018). The Race between Man and Machine: Implications of Technology for Growth, Factor Shares, and Employment. American Economic Review, 108(6), 1488-1542.
9. Brynjolfsson, E., & McAfee, A. (2014). The Second Machine Age: Work, Progress, and Prosperity in a Time of Brilliant Technologies. W. W. Norton & Company.

Websites and Tools

10. World Economic Forum. (2023). The Future of Jobs Report 2023. https://www.weforum.org/reports/the-future-of-jobs-report-2023
11. Visual Studio Code — Five Server Extension. (2024). Live Server Extension for VS Code. https://marketplace.visualstudio.com/items?itemName=yandeu.five-server
12. W3Schools. (2024). HTML5, CSS3, JavaScript Tutorials. https://www.w3schools.com

---

— End of Report —